

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño	Definir una estructura modular, escalable y mantenible que distribuya responsabilidades entre UI, API, dominio, datos e infraestructura, minimizando acoplamiento y permitiendo trazar los UC de tareas/KPIs a módulos claros.
Orientación de la decisión	Adoptar arquitectura en capas con paquetes: Frontend (React): pages, components, api, state, hooks, utils, styles. Backend (Django+DRF): API (DRF) (routers/viewsets/serializers), Apps de dominio (users, projects, tasks, kpis, reporting, notifications), Core (servicios de dominio), Config/Admin/Integrations. Data: schema, migrations. Infra: auth (JWT), cache (Redis), broker (Celery), storage (S3/MinIO), mail (SMTP). Dependencias permitidas: Frontend → API, API → Apps → Core → ORM, Backend → Data/Infra.
Supuestos	- Proyecto en transición diseño/desarrollo con entregables iterativos. - Backend en Django + DRF, frontend en React, BD MySQL. - UC prioritarios: crear/asignar/editar tareas, registrar tiempo, KPIs, dashboard, reportes, notificaciones.
Restricciones	- Tecnología definida (Django/DRF, React, MySQL). - Tiempo de curso: monolito modular (no microservicios). - Seguridad con JWT; almacenamiento de reportes en S3/MinIO; colas con Celery (cuando aplique).
Requerimientos relacionados	UC-01, UC-02, UC-03, UC-04, UC-05, UC-06, UC-07, UC-08, UC-09, UC-10, UC-11, UC-12, UC-13, UC-14, UC-15, UC-16, UC-17, UC-18.
Alternativas consideradas	Alternativa 1: Publicador–Suscriptor (event-driven entre módulos). Alternativa 2: Cliente–Servidor 2 capas (UI ↔ BD vía lógica mínima). Alternativa 3: Capas con paquetes de dominio (propuesta).
Evaluación de alternativas	Alternativa 1: Pub–Sub Ventajas: desacoplamiento, extensible para eventos (notifs, reportes). Desventajas: complejidad operativa (broker everywhere), difícil trazabilidad para el curso. Alternativa 2: Cliente–Servidor 2 capas Ventajas: desempeño simple, baja latencia directa. Desventajas: baja modularidad/extensibilidad, mezcla de responsabilidades, difícil testing.

	Alternativa 3: Capas con paquetes (seleccionada) • Ventajas: modularidad, escalabilidad, trazabilidad UC→módulo, alineada a DRF/React y al ER. • Desventajas: ligera sobrecarga inicial (más carpetas y contratos).
Alternativa seleccionada y Justificación	Arquitectura por capas con paquetes de dominio. Centraliza endpoints en API (DRF), lógica en Core/Apps, acceso único a datos vía ORM y consumo de Infra solo desde Backend. Maximiza cohesión, minimiza acoplamiento y mapea de forma directa los UC (p. ej., UC-01/02/04 en tasks; UC-06/09 en kpis; UC-07/08 en reporting).
Decisiones relacionadas	• Decisión 2 (Persistencia/ORM y modelo ER). • Definir políticas de autorización/roles (users/permissions). • Estándares de API (paginación, filtros, errores).
Comentarios	Se incluye un orden de implementación recomendado: [1] Data (schema+migraciones) → [2] Config (settings/urls/JWT) → [3] Users/Auth → [4] Projects → [5] Tasks+Core → [6] Frontend base → [7] KPIs → [8] Reporting → [9] Notifications.